

Comprehensive Notes on HTML and CSS with Structured Theory and Detailed Code Explanation

1. Introduction to HTML and CSS

What is HTML?

HTML (HyperText Markup Language) is the standard language used to create and structure web pages. It consists of a series of **elements (tags)** that tell the browser how to display content. HTML provides the basic framework, which can then be styled using CSS.

What is CSS?

CSS (Cascading Style Sheets) is a styling language used to design and enhance the presentation of web pages. It allows developers to control **colors, fonts, layout, spacing, and responsiveness**.

How HTML and CSS Work Together?

- HTML structures the web page with elements such as headings, paragraphs, images, and links.
 - CSS adds design elements like colors, backgrounds, font sizes, and layouts.
 - Combining both ensures a visually appealing and well-structured webpage.
-

2. HTML Structure and Explanation

index.html (Main Webpage Structure)

Purpose:

The `index.html` file is the main entry point of a website. It contains text, images, links, and references to external resources like CSS files.

Complete Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<link rel="stylesheet" href="style.css"/>
<style>
  body {
    color: brown;
  }
</style>
</head>
<body>
  <h1>This is Jason Beryl</h1>
  <h2 style="background-color: blue;">I am currently doing an internship</h2>
  <h3>in accelerlab, Moodbidri</h3>

  

  <a href="mailto:jasonberyl000@gmail.com">Send me an email</a>
</body>
</html>
```

Code Explanation:

- `<!DOCTYPE html>` → Declares the document type as HTML5.
- `<html lang="en">` → Defines the root HTML element and sets the page language to English.
- `<head>` → Contains metadata such as character encoding, viewport settings, and external CSS links.
- `<meta charset="UTF-8">` → Ensures support for special characters and multiple languages.
- `<meta name="viewport" content="width=device-width, initial-scale=1.0">` → Makes the page **responsive** to different screen sizes.
- `<title>Document</title>` → Sets the webpage title (visible on the browser tab).
- `<link rel="stylesheet" href="style.css"/>` → Links an external CSS file (`style.css`) for styling.
- `<style>` → Contains internal CSS that applies a **brown text color** to the webpage.
- `<body>` → Contains the main content displayed on the webpage.
- `<h1>` , `<h2>` , `<h3>` → Defines different heading sizes (largest to smallest).
- `` → Displays an image with a **placeholder source URL**.
- `<a href="mailto:jasonberyl000@gmail.com">Send me an email</a>` → Creates an email link that opens the user's mail client.

3. CSS Styling and Explanation

style.css (External Styling)

Purpose:

The `style.css` file controls the appearance of the webpage by defining styles for different HTML elements.

Complete Code:

```
body {  
  background-color: green;  
}
```

Explanation:

- `body {}` → Selects the entire webpage.
- `background-color: green;` → Sets the background color of the page to green.

Using an **external CSS file** helps in maintaining cleaner code and makes styling reusable across multiple HTML pages.

4. Link Styling and States

text.html (Hyperlink Styling)

Purpose:

This CSS file modifies how links appear in different states.

Complete Code:

```
a:link { color: green; text-decoration: none; }  
a:visited { color: pink; }  
a:hover { color: red; text-decoration: underline; }  
a:active { color: yellow; }
```

Explanation:

- `a:link` → Styles links **before they are clicked** (green color, no underline).
- `a:visited` → Styles links **after they have been clicked** (pink color).

- `a:hover` → Styles links when the user **hovers** over them (red color, underlined).
 - `a:active` → Styles links **while they are being clicked** (yellow color).
-

5. Understanding HTML Tables

table.html (Table Structure)

Purpose:

This file contains an HTML table used to organize data into rows and columns.

Complete Code:

```
<table>
  <tr>
    <th colspan="2">Name</th>
    <th>Roll No</th>
  </tr>
  <tr>
    <th rowspan="2">Age</th>
    <td>20</td>
  </tr>
  <tr>
    <td>2</td>
  </tr>
</table>
```

Explanation:

- `<table>` → Defines a table.
 - `<tr>` → Defines a **table row**.
 - `<th>` → Defines a **header cell** (bold and centered by default).
 - `<td>` → Defines a **data cell**.
 - `colspan="2"` → Merges **two columns into one**.
 - `rowspan="2"` → Merges **two rows into one**.
-

6. Additional Notes and Best Practices

HTML Best Practices:

✅ Use **semantic HTML** (e.g., `<header>`, `<nav>`, `<footer>` instead of `<div>` for clarity). ✅
Keep **content and styling separate** (use external CSS instead of inline styles). ✅ Use **alt attributes** for images to improve accessibility and SEO. ✅ Write **clean, readable code** by properly indenting and structuring elements.

CSS Best Practices:

✅ Use **external stylesheets** for reusable and maintainable styling. ✅ Apply **relative units** (% , `em`, `rem`) instead of fixed pixel values for responsiveness. ✅ Minimize use of `!important` to avoid overriding unintended styles.

Advanced Topics to Explore:

🚀 **CSS Grid & Flexbox**: For modern layouts. 🚀 **Media Queries**: For making pages mobile-friendly.
🚀 **CSS Animations & Transitions**: For dynamic user experience. 🚀 **JavaScript**: To add interactivity like form validation and event handling.

7. Conclusion

This lesson provided an in-depth understanding of **HTML structure**, **CSS styling**, **hyperlink states**, and **tables**. Mastering these concepts is crucial for building interactive and visually appealing web pages.

Next Steps:

- Experiment with **different CSS properties** and layouts.
- Learn about **responsive design** using media queries.
- Start integrating **JavaScript** for dynamic functionalities.

By mastering HTML and CSS, you are taking a significant step towards becoming a proficient **front-end web developer**. Keep practicing! 🚀